Identifying SPOFs

ask "what if this dies?" for every box and arrow

Watch the 3-min video

In one line: walk every box, arrow and external dependency and ask "what if this dies?"; anything whose loss stops the system is a SPOF, removed with redundancy across zones, automatic failover, graceful degradation, and timeouts with circuit breakers.

1 Redundant servers, still down

Your app runs on 20 servers across three zones. Fully redundant? Then your login provider goes down and nobody can sign in. Not one of your servers failed. A single point of failure was hiding in a dependency nobody drew on the diagram.

A **single point of failure (SPOF)** is any component whose failure takes down the whole system, *or a critical feature*. It isn't only servers. It can be a database primary, a load balancer, a network link, a config store, an expiring certificate, one region, or one person who knows how to deploy.

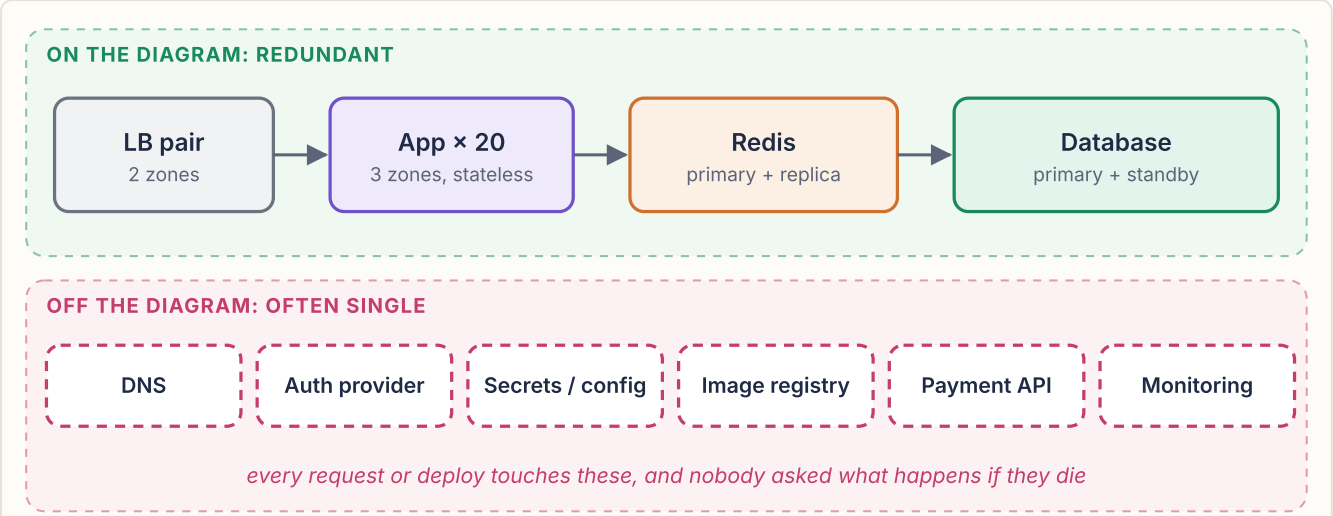


Figure 1. The drawn system is redundant. The dependencies around it, each used by every request or every deploy, are where outages hide.

2 The method: box by box, arrow by arrow

Walk your diagram one box at a time and ask:

1. **What happens if this dies?**
2. Is there another copy that takes over?
3. How quickly, and is it automatic or by hand?

If the answer is "the whole system stops", you have found a SPOF. Do the same for **every arrow**, because links fail too: a single VPN tunnel, one cross-region link, one NAT gateway.

3 The dangerous ones are off the diagram

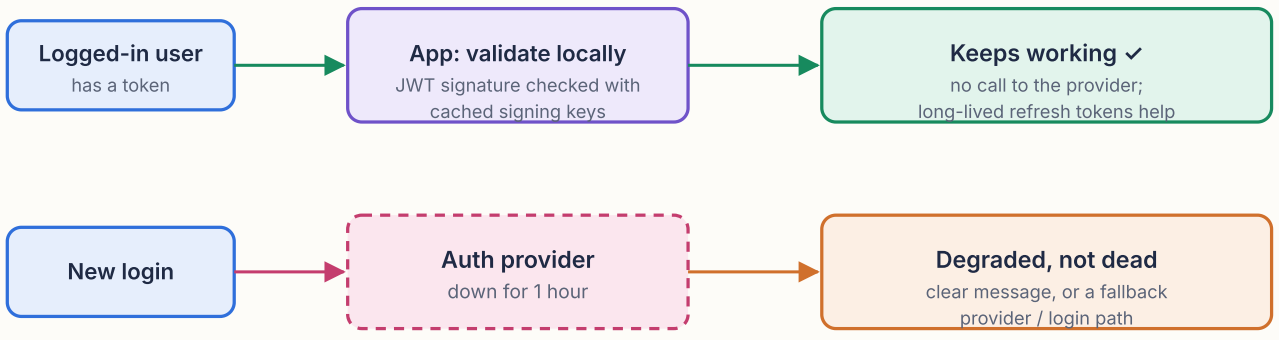
DNS, your identity or auth provider, secrets and config stores, the container registry, a third-party payment API, even your monitoring. Build a **dependency map**: list every external system the critical path

touches, and treat each one as a box you must question.

WORKED EXAMPLE: THE AUTH PROVIDER GOES DOWN

Every login calls an external auth provider. It goes down for an hour and nobody can log in, even though all your servers are healthy. Mitigations:

- **Keep logged-in users working:** validate tokens locally with **cached signing keys**, so each request does not call the provider.
- **Longer-lived refresh tokens**, so sessions survive a short outage.
- A **clear degraded message** for new logins instead of a timeout.
- For critical systems, a **second provider** or a fallback login path.



your 20 servers are healthy in both rows; only the dependency failed

Figure 2. Local token validation turns an auth outage from "nobody can use the app" into "only new logins are affected".

4 The general fixes



Figure 3. The same four tools remove or soften almost every SPOF.

Add **redundancy**: run at least two of everything, across zones. Add **automatic failover** with health checks, so recovery doesn't wait for a human. **Degrade gracefully**: if recommendations fail, still show the page. And add **timeouts and circuit breakers**, so a dead dependency fails fast instead of hanging every request and exhausting your thread pools.

5 Walk it out loud in an interview

Component	If it dies...	How it survives
Load balancer	No traffic gets in	A redundant pair, or a managed load balancer across zones
App servers	Capacity drops	Stateless, many instances; others absorb the load
Cache	Every read hits the database	Replicas, and a database sized to survive a cold cache
Database	No writes, maybe no reads	Primary with a standby and automatic failover
DNS, auth	Nobody finds you, or nobody logs in	Second DNS provider; local token validation, fallback login

You can **accept a SPOF** when removing it isn't worth the cost, say a single-region internal tool, but say so explicitly and name the risk.

COMMON MISTAKES

- Only checking servers, never arrows or external dependencies.
- A standby with manual failover at 3 a.m.: the outage lasts as long as it takes to wake someone.
- A cache that is redundant, but a database that collapses when the cache is cold.
- No timeouts: one hung dependency ties up every thread and takes healthy services down with it.
- Silently accepting a SPOF instead of naming it and its cost.

6 Key terms

SPOF: a component whose loss stops the system or a critical feature

Automatic failover: health checks promote a backup without a human

Circuit breaker: stops calling a failing dependency for a while

Dependency map: every external system the critical path touches

Graceful degradation: keep core features when extras fail

Cold cache: an empty cache after a restart or failover

7 Interview questions

Q1. How do you find single points of failure? BASIC

Walk every component and arrow and ask what happens if it dies, is there a copy, and how fast it takes over. Include hidden dependencies: DNS, auth, secrets and config stores, registries, third-party APIs.

Q2. How do you remove them? BASIC

Redundancy across zones (at least two of everything), automatic failover with health checks, graceful degradation, and timeouts with circuit breakers.

Q3. Is a SPOF always a server? BASIC

No. It can be a network link, a config store, an expiring certificate, one region, or one person who knows how to deploy.

Q4. The external auth provider is down for an hour. What still works? SCENARIO

If tokens are validated locally with cached signing keys, logged-in users keep working; refresh tokens extend sessions. New logins get a clear degraded message, or a second provider.

Q5. Your Redis cluster fails over and the cache is empty. What happens? SCENARIO

Every read hits the database at once. If it was sized for a 90% hit rate, it suddenly sees 10× the reads. Size the database for a cold cache, warm the cache, or rate-limit until it refills.

Q6. A recommendations service hangs. Why does the whole page time out? SCENARIO

No timeout: each request waits, threads pile up and the pool runs dry. Add a timeout (say 200 ms), a circuit breaker, and render the page without recommendations.

Q7. Your monitoring runs in the same region as the app. Problem? TRICKY

Yes. A regional outage blinds you exactly when you need it, and alerts may never fire. Run alerting or external probes from outside the failure domain.

Q8. When is it fine to keep a SPOF? TRICKY

When removing it costs more than the outage risk, like a second auth provider for an internal tool. Say so explicitly, estimate the impact, and have a recovery plan.

Next: SD-G-001 · Load Balancer Fundamentals backendroadmap.com